

Veri Yapıları ve C# Programlama

Görsel Programlama - Hafta 13

Veri Yapıları ve C# Programlama

Bu sunumda, C# programlamada temel veri saklama yöntemlerinden biri olan dizileri (Arrays) derinlemesine inceleyeceğiz.

Veri yapıları, büyük miktardaki veriyi etkili bir şekilde organize etmek ve yönetmek için kritik öneme sahiptir.

Haftanın ana odağı, dizilerin tanımlanması, bellekte saklanması ve döngülerle işlenmesidir.



Haftalık Konulara Genel Bakış

Teorik Temeller: Dizinin tanımı ve indeksleme mantığı.

C# Söz Dizimi: Dizi tanımlama ve anahtar kelimeler.

Döngü Entegrasyonu: Dizileri işlemek için for ve foreach kullanımı.

Proje Uygulamaları: Öğrenci listesi yönetimi ve dizideki en büyük/en küçük elemanı bulma algoritmaları.



Dizinin Temel Tanımı

Dizi (Array), aynı veri tipine sahip birden fazla elemanı tek bir değişken adı altında gruplayan, sabit boyutlu bir veri yapısıdır.

Örneğin, 10 farklı öğrenci adını saklamak yerine, tek bir 'ogrenciler' dizisi kullanırız.

Diziler, bellekte ardışık (contiguous) konumlarda saklanır, bu da verilere hızlı erişim sağlar.

İndeksleme Mantığı

Dizilerdeki her bir eleman, konumunu belirten bir indeks numarası ile erişilir.

Programlama dillerinin çoğunda olduğu gibi, C# dizilerinde de indeksleme ****0'dan**** başlar.

Eğer bir dizinin boyutu N ise, son elemanın indeksi N-1 olacaktır.

Örnek: 'diziAdi' ilk elemanı temsil eder.

Bellek Yönetimi: Dizilerin Avantajı

Diziler tanımlandığında, tüm elemanları depolayacak kadar ardışık bir bellek bloğu tahsis edilir.

Bu ardışıklık, dizinin herhangi bir elemanına doğrudan ($O(1)$ zaman karmaşıklığı ile) erişim imkanı tanır.

Ancak bu yapı, dizinin boyutunun başlangıçta sabitlenmesini zorunlu kılar.

C# Dizi Tanımlama Söz Dizimi (Syntax)

Dizi tanımlamanın iki temel yolu vardır:

1. Boyut Belirleyerek:

```
'string[] diziAdi = new string[boyut];'
```

Bu, belirli bir sayıda (örneğin 10) boş (varsayılan değerde) eleman tutacak bir dizi oluşturur.

2. Başlangıç Değeri Atayarak (Implicit Size):

```
'int[] sayilar = { 1, 5, 10 };'
```

Derleyici, süslü parantezler içindeki eleman sayısına göre boyutu otomatik olarak belirler.



Dizilerin Döngülerle İşlenmesi

Dizilerdeki birden fazla elemana erişmek ve onları işlemek için döngüler hayati öneme sahiptir.

'for' Döngüsü: İndeks numarası üzerinden kontrol sağlar ve belirli bir aralıktaki elemanları işlemek için idealdir.

'foreach' Döngüsü: İndeksle ilgilenmeden, dizideki her bir elemanı sırayla ziyaret etmek için kullanılır. Özellikle sadece okuma işlemlerinde kullanışlıdır.

Temel Form Kontrolleri

Windows Forms uygulamalarında, dizilerle etkileşim kurmak için belirli UI (Kullanıcı Arayüzü) kontrolleri kullanılır.

ListBox: Dizinin içeriğini liste halinde, kolay okunabilir bir formatta göstermek için en ideal kontroldür.

Button: Kullanıcı eylemlerini (örneğin 'Ekle', 'Sayı Üret', 'Bul') tetiklemek için kullanılır.

TextBox ve Label: Veri girişi almak ve sonuçları göstermek için gereklidir.



Proje 1: Öğrenci Listesi Oluşturma

Amaç: Kullanıcının girdiği öğrenci adlarını bir 'string' dizisine kaydetmek ve ardından bunu bir ListBox kontrolünde göstermektir.

Bu proje, diziye veri ekleme, indeks takibi yapma ve dizi kapasitesini kontrol etme becerilerini pekiştirir.



Proje 1: Kullanıcı Arayüzü (UI) Yapısı

TextBox (txtOgrenciAd): Kullanıcının yeni öğrenci adını gireceği metin alanı.

Button ('Ekle'): Diziye ekleme işlemini başlatan düğme.

ListBox (listBox1): Diziye eklenen öğrencilerin listelendiği kontrol.

Bu basit arayüz, veri girişi, işleme ve çıktıyı aynı anda yönetmemizi sağlar.

Proje 1: Global Değişkenlerin Tanımlanması

Uygulama genelinde erişilebilir olması gereken iki temel global değişken tanımlanır:

1. Öğrenci Dizisi:

```
'string[] ogrenciler = new string;'
```

Bu, 10 kişilik sabit kontenjana sahip bir string dizisidir.

2. İndeks Sayacı:

```
'int ogrenciIndex = 0;'
```

Bu değişken, dizinin hangi boş pozisyonuna ekleme yapılacağını takip eder ve 0'dan başlar.

P1: 'Ekle' Butonuna Tıklama Olayı

Butona tıklandığında çalışacak kod bloğu, diziye yeni bir eleman ekleme sürecini başlatır.

İlk adım, dizinin kapasitesinin dolup dolmadığını kontrol etmektir. Sabit boyutlu dizilerde bu kontrol kritiktir.



Pl: Kapasite Kontrolü

Diziye ekleme yapmadan önce, mevcut öğrenci indeksinin (kaçıncı elemanı eklediğimizi gösteren) dizinin boyutundan küçük olup olmadığını kontrol ederiz:

'if (ogrenciIndex < 10)'

Bu koşul doğruysa (yani kontenjan varsa), ekleme işlemine devam edilir.

Yanlışsa, dizinin dolduğu anlamına gelir ve kullanıcıya uyarı verilir.

P1: Diziye Veri Atama

Kontenjan uygunsa, TextBox'taki metin (yeni öğrenci adı) mevcut 'ogrenciIndex' pozisyonuna atanır:

```
'ogrenciler[ogrenciIndex] = txtOgrenciAd.Text;'
```

Bu işlemle, veriyi kalıcı olarak diziye kaydetmiş oluruz.

P1: ListBox'a Ekleme

Veri diziye kaydedildikten hemen sonra, kullanıcı arayüzünü güncellemek gerekir.

Eklenen öğrenci adı, ListBox kontrolüne de eklenir:

```
'listBox1.Items.Add(ogrenciler[ogrenciIndex]);'
```

Unutmayın, ListBox'a eklemek dizinin içeriğini değiştirmez, sadece dizideki elemanın görünür olmasını sağlar.

P1: İndeks Artırma İşlemi

Ekleme ve UI güncellemesi tamamlandıktan sonra, bir sonraki öğrencinin kaydedileceği pozisyonu işaretlemek zorundayız:

```
'ogrenciIndex++;'
```

Bu artış, bir sonraki buton tıklamasında verinin dizinin hemen arkasındaki boş hücreye kaydedilmesini garantiler.



P1: Giriş Alanını Temizleme

Başarılı bir ekleme işleminden sonra, kullanıcının yeni bir ad girmesini kolaylaştırmak için TextBox içeriği temizlenir:

```
'txtOgrenciAd.Clear();'
```

Bu, iyi bir kullanıcı deneyimi (UX) uygulamasıdır.

Pl: Kontenjan Dolu Durumu

Eğer 'if (ogrenciIndex < 10)' koşulu sağlanmazsa (yani 'ogrenciIndex' 10 veya daha büyükse), 'else' bloğu çalışır.

Bu durumda, dizi doludur ve kullanıcıya uygun bir mesaj ("Kontenjan Dolu" uyarısı) gösterilmelidir.

Bu, bir dizinin sabit boyutlu olmasının getirdiği bir sınırlamadır.

P1: Kod Akışı Özeti (Özet Kriterler)

1. Buton Tıklaması
2. İndeks < Kapasite kontrolü
3. Kontenjan varsa: Veri diziye atanır.
4. ListBox güncellenir.
5. İndeks artırılır.
6. TextBox temizlenir.
7. Kontenjan yoksa: Uyarı mesajı gösterilir.

Akademik Çıkarım: Dizilerin Sınırlılığı

Öğrenci listesi örneğinde gördüğümüz gibi, dizilerde kapasite önceden belirlenmek zorundadır (örneğimizde 10).

Eğer ekleme sırasında kontenjan biterse, yeni bir dizi oluşturup verileri kopyalamak gerekir.

Bu sınırlama, C# dilinde daha esnek olan 'List<T>' gibi koleksiyon sınıflarını kullanma ihtiyacını doğurur.



Proje 2: Dizideki Arama Algoritması

Amaç: Bir 'int' dizisi üzerinde döngü ve koşul (if) kullanarak, dizideki en büyük veya en küçük sayısal elemanı bulmak.

Bu proje, temel döngüsel algoritma mantığını ve karşılaştırma operatörlerinin kullanımını gösterir.



P2: Kullanıcı Arayüzü (UI) Gereksinimleri

ListBox (listBox1): Üretilen sayıların gösterildiği alan.

Button ('Sayı Üret'): Diziyi rastgele sayılarla dolduran düğme.

Button ('En Büyüğü Bul'): Arama algoritmasını başlatan düğme.

Label (labelSonuc): Algoritma sonucunun (en büyük/küçük sayı) gösterildiği etiket.

P2: Global Dizi Tanımlaması

Bu proje için bir tam sayı (integer) dizisi tanımlanır:

```
'int[] sayilar = new int;'
```

Bu, 20 adet tam sayıyı depolayacak kapasiteye sahip olduğumuz anlamına gelir.

P2: 'Sayı Üret' Butonu Kod Akışı

Bu buton, diziyi analize hazır hale getirmek için 20 adet rastgele sayıyla doldurur.

İlk adım: 'Random' sınıfı örneği oluşturulur: `Random rnd = new Random();`

İkinci adım: ListBox temizlenir: `listBox1.Items.Clear();`

P2: Sayı Üretim Döngüsü

Dizinin tüm elemanlarını gezmek için bir 'for' döngüsü kullanılır:

```
'for (int i = 0; i < sayilar.Length; i++)'
```

Döngü içinde rastgele sayı ataması yapılır:

```
'sayilar[i] = rnd.Next(1, 1000);' (1 ile 999 arasında rastgele sayılar üretilir).
```

P2: Üretilen Sayıların Gösterimi

Sayı diziye atandıktan hemen sonra, kullanıcının hangi sayılar üzerinde işlem yapılacağını görmesi için ListBox'a eklenir:

```
'listBox1.Items.Add(sayilar[i]);'
```

Bu işlem, döngü her çalıştığında tekrarlanır ve 20 sayının tamamı eklenir.

P2: 'En Büyüğü Bul' Butonu Mantığı

Bu işlem, dizideki her elemanı tek tek kontrol ederek, şu ana kadar görülen en büyük değeri sürekli güncelleyen bir algoritma gerektirir.

Algoritma, karşılaştırma ve atama işlemlerine dayanır.

P2: Başlangıç Kabulü (Tohum Değer)

Aramaya başlamadan önce, en büyük sayıyı geçici olarak tutacak bir değişken tanımlanır:

'int enBuyuk = sayilar;'

****Neden İlk Eleman?*** Dizinin herhangi bir elemanını (genellikle ilk eleman) başlangıç değeri olarak kabul etmek, tüm diğer elemanların bu değere göre karşılaştırılabilmesini sağlar.

P2: Arama Döngüsü Kurulumu

Başlangıç değerini 'sayılar' olarak aldığımız için, döngüyü dizinin ikinci elemanından (indeks 1) başlatarak gereksiz bir karşılaştırmadan kaçınırız:

```
'for (int i = 1; i < sayilar.Length; i++)'
```

Döngü, dizinin sonuna kadar (boyut-1) devam eder.

P2: Karşılaştırma İşlemi

Döngü içindeki ana mantık, o anki elemanın ('sayilar[i]') mevcut en büyük değerden ('enBuyuk') daha büyük olup olmadığını kontrol etmektir:

'if (sayilar[i] > enBuyuk)'

P2: En Büyük Değeri Güncelleme

Eğer koşul doğruysa (yani mevcut eleman, şu ana kadarki en büyükten daha büyükse), 'enBuyuk' değişkeni güncellenir:

```
'enBuyuk = sayilar[i];'
```

Bu atama, dizinin geri kalan elemanlarının bu yeni büyük değere göre karşılaştırılmaya devam etmesini sağlar.



P2: Lineer Arama Algoritması (Örnek Akış)

Dizi: { 5, 12, 3, 8 }

1. Başlangıç: 'enBuyuk = 5' (indeks 0)
2. $i=1$ (12): $12 > 5$? Evet. 'enBuyuk = 12'
3. $i=2$ (3): $3 > 12$? Hayır.
4. $i=3$ (8): $8 > 12$? Hayır.

Sonuç: En büyük sayı 12.

P2: En Küçük Elemanı Bulma Mantığı

En küçük elemanı bulma algoritması, en büyük elemanı bulma algoritmasının neredeyse aynısıdır.

Tek fark, karşılaştırma operatörünün tersine çevrilmesidir:

Başlangıç: `'int enKucuk = sayilar;'`

Karşılaştırma: `'if (sayilar[i] < enKucuk)'`

Eğer mevcut eleman, şu anki en küçükten daha küçükse, 'enKucuk' değeri güncellenir.

Gelişmiş Dizi Kavramları: Çok Boyutlu Diziler

Tek boyutlu diziler (vektörler) öğrendik. Ancak C# iki veya daha fazla boyutlu dizileri de destekler.

İki boyutlu diziler (matrisler), satır ve sütun indeksleri kullanılarak verileri tablo şeklinde saklamak için idealdir.

Tanımlama Örneği: `int[,] matris = new int;` (3 satır, 5 sütun).

Çok Boyutlu Diziler: Erişim ve Döngü

Çok boyutlu dizilerde her bir elemana erişmek için iç içe döngüler (nested loops) kullanılır.

Dış döngü genellikle satır indeksini, iç döngü ise sütun indeksini kontrol eder.

Erişim Örneği: 'matris[satirIndex, sutunIndex]'

Bu yapı, bilimsel hesaplamalarda ve oyun geliştirmede (ızgara sistemleri) yaygın kullanılır.



Dizilere Alternatif: Koleksiyonlar

Dizilerin sabit boyutlu olma kısıtlamasını aşmak için C# Collections Framework'teki 'List<T>' (Generic List) yapısı kullanılır.

Listeler, dinamik olarak büyüyebilir ve küçülebilir; bu da Proje 1'deki kontenjan kontrolü ihtiyacını ortadan kaldırır.

Listelerin performansı, diziler kadar hızlı olmasa da esneklikleri nedeniyle çoğu uygulamada tercih edilir.



Foreach Döngüsünün Detayları

Foreach döngüsü, sadece dizinin içeriğini okuma amaçlı gezmek istediğimizde 'for' döngüsüne göre daha temiz ve okunabilir bir alternatif sunar.

Söz Dizimi: 'foreach (tip degiskenAdi in diziAdi)'

Önemli Not: 'foreach' döngüsü içinde dizinin elemanlarını doğrudan değiştirmek mümkün değildir; indeks bilgisi yoktur.

Dizide Arama Karmaşıklığı (Complexity)

Proje 2'de uyguladığımız en büyük/en küçük elemanı bulma algoritması, Lineer Arama (Sequential Search) olarak bilinir.

Bu algoritmanın zaman karmaşıklığı $O(N)$ 'dir (N: Dizi Boyutu).

Başka bir deyişle, dizinin boyutu iki katına çıkarsa, arama süresi de yaklaşık olarak iki katına çıkar. Çünkü en kötü durumda tüm elemanları gezmek zorundayız.



Değer Tipleri (Value Types) ve Referans Tipleri (Reference Types)

C#'ta 'int' gibi değer tiplerinden oluşan diziler (Proje 2), elemanları doğrudan belleğin Stack bölgesinde saklayabilir.

'string' gibi referans tiplerinden oluşan diziler (Proje 1) ise, dizinin kendisi Stack'te olsa bile, elemanların içerdiği veriler (string değerleri) Heap bölgesinde saklanır. Dizi hücreleri sadece Heap'teki adresleri tutar.

P1 Detay: Hata Kontrolünün Önemi

Eğer Proje 1'de kapasite kontrolünü ('if (ogrenciIndex < 10)') yapmasaydık, 11. eklemede program 'IndexOutOfRangeException' hatası vererek çökerdi.

Bu, programlamada hata yakalama (exception handling) ve sınır kontrolünün ne kadar önemli olduğunu gösterir.



P2 Detay: Random Sınıfının Tekrar Kullanımı

Kodumuzda 'Random rnd = new Random();' ifadesini her buton tıklamada çalıştırmak yerine, genellikle Random nesnesini global olarak tanımlamak daha iyi bir uygulamadır.

Her yeni çağrıda yeni bir Random nesnesi oluşturulursa, çok kısa aralıklarla çağrıldığında aynı 'seed' (çekirdek) değeri kullanılarak aynı sayı dizisi üretilebilir.



Örnek Dizi İşlemleri: Diziyi Ters Çevirme

Dizileri kullanarak yapabileceğimiz yaygın bir işlem de elemanları ters çevirmektir.

Bu, iki işaretçi (pointer) kullanarak, dizinin başlangıcından ve sonundan başlayıp elemanları birbiriyle takas ederek yapılır.

Bu tür indeks manipülasyonları, döngü mantığının derinlemesine anlaşılmasını gerektirir.

Dizi Verilerinin Sıralanması (Sorting)

Dizideki en büyük veya en küçük elemanı bulmak bir arama problemidir. Daha karmaşık problemlerden biri ise diziyi sıralamaktır (küçükten büyüğe veya tam tersi).

Temel sıralama algoritmaları: Bubble Sort, Selection Sort, Insertion Sort.

Bu algoritmalar genellikle iç içe döngüler kullanır ve $O(N^2)$ zaman karmaşıklığına sahiptir.

C# Standart Kütüphanesi ve Diziler

C#, dizilerle çalışmayı kolaylaştırmak için yerleşik 'System.Array' sınıfı ve metotları sunar.

Örnek Metotlar: 'Array.Sort()' (Diziyi hızlıca sıralar), 'Array.Reverse()' (Diziyi ters çevirir), 'Array.IndexOf()' (Elemanın indeksini bulur).

Bu metotlar, Proje 2'deki gibi manuel algoritmalar yazma zorunluluğunu azaltır.

Dizilerde Arama Performansının İyileştirilmesi

Eğer bir dizi sıralıysa, lineer arama ($O(N)$) yerine çok daha hızlı olan İkili Arama (Binary Search) kullanılabilir.

İkili Arama, $O(\log N)$ karmaşıklığına sahiptir.

Mantığı: Diziyi sürekli ikiye bölerek hedefin hangi yarıda olduğunu kontrol etmeye dayanır.

String Dizileri ve Metin İşleme

Proje 1'de string dizisi kullandık. Stringler, C#'ta karakter dizileridir ve dizilerle birlikte metin analizi için güçlü araçlardır.

Metin parçalama, birleştirme, belirli kelime arama gibi işlemler string dizileri üzerinde döngüler ve koşullar kullanılarak yapılabilir.



Dizilerde Verimlilik ve Bellek İzleri

Diziler, bellekte sabit yer kapladıkları için tahsis edildikten sonra bellekteki parçalanmayı (fragmentation) azaltır.

Büyük boyutlu dizilerin (özellikle referans tiplerinin) Heap alanında ayrılması, Garbage Collector'ın (Çöp Toplayıcı) yükünü artırabilir. İyi bellek yönetimi kritik öneme sahiptir.



Özet: Diziler ve C# Anahtar Kelimeleri

Tanım: Aynı türden verileri indeksle saklama.

Tanımlama: 'new type[boyut]' veya '{değerler}'.

Erişim: 'diziAdi[index]'.

İşleme: 'for' veya 'foreach' döngüleri ile kombinasyon.

Uygulama Alanları: Sabit veri kümeleri, algoritmik arama ve sıralama (Proje 2), basit liste yönetimi (Proje 1).



Kapanış ve Gelecek Konular

Bu hafta dizilerin temellerini ve iki pratik C# uygulamasını gördük.

Bir sonraki adım, dizilerin sabit boyutlu sınırlamasını aşan dinamik veri yapılarını incelemek olacaktır.

Örneğin, esneklik ve gelişmiş metotlar sunan List, Dictionary ve Queue gibi koleksiyon sınıfları ele alınacaktır.



Veri Yapıları ve C# Programlama

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

